# A Crash-Proof Team of Coding Agents

*Claude Code remembers the conversation; Temporal remembers the job. Kill the worker mid-task and the agent finishes anyway: the same session, resumed from its last heartbeat, no completed checkpoint required. First, that makes one job crash-proof. Then many such jobs, each a single-purpose durable step with its own mandate, compose into a governed team.*


---

There is a moment in every infrastructure demo where you stop trusting the slides and ask the presenter to pull the plug. We opened this project by pulling it ourselves. The setup was ordinary on purpose. A worker process was running a headless coding agent (headless meaning one command in a terminal, no chat window) on a small test-first Python task. The agent was minutes in, actively writing files, nothing finished, nothing saved anywhere. We killed the worker's entire process tree with an unblockable signal. No goodbye, no flush, no checkpoint.

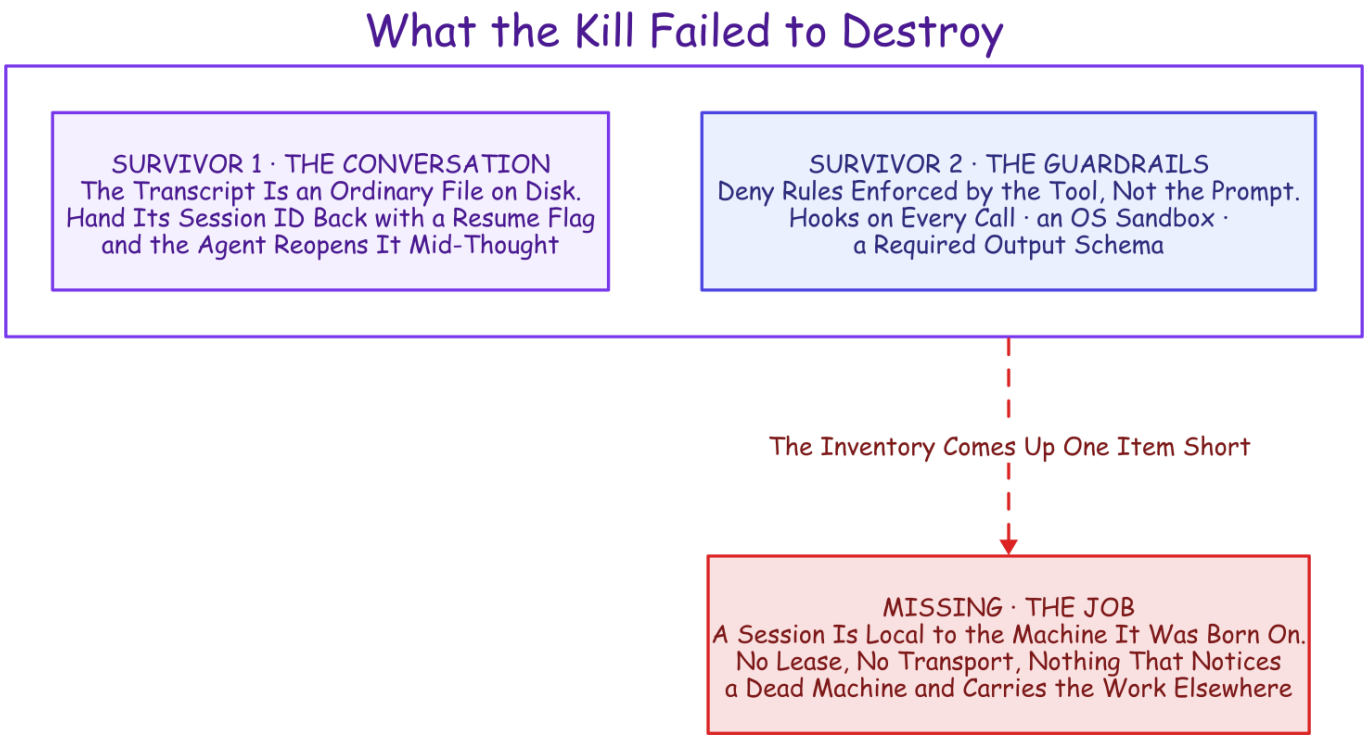
For about two minutes, nothing happened at all. Then a different process, on a restarted worker, noticed the silence. It picked up the same half-finished conversation and ran it to completion: the same session, remembering everything it had learned. The surviving attempt's bill was four cents. Keep the four cents; the receipt shows up later. And the staged kill is only the legible version of what happens uninvited every week: an API overload, a rate limit, a deploy that restarts the worker.

How that session survived is the first half of this article. The bigger half is what survival buys once you have it: a durable team. The work splits into single-purpose durable jobs. Only one kind of step runs Claude Code itself, and it does all the judgment work: building the feature, reviewing the diff, resolving the conflict. The rest carry the delivery pipeline around it. That team carried a filed GitHub issue all the way to a merged pull request, resolving a real merge conflict along the way, with no human decision in the loop. One mechanical asterisk rides on that sentence, spelled out where the story is told.

*The fine print, up front: one engineer ran everything here, and the "we" is editorial. Every measured run used `haiku`, Claude's cheap fast tier, in July and August 2026; the run counts are single-digit, so every number is a point estimate that traces to a results file in the project's evidence archive. Dollar figures are haiku-priced; a stronger tier scales them up. And "crash-proof" here means process and worker crashes, the staged kill included; a machine that never comes back is an admitted gap, covered in What This Doesn't Solve. So is the merge gate: it turns on an agent's own tests-pass verdict, a signal a companion measured wrong three times in ten.*

## The Agent Remembers the Conversation

Start with the crime scene, and take inventory of what the kill failed to destroy.




*The inventory. Two survivors: the transcript on disk, reopenable by its session ID, and the guardrail harness, enforced by the tool rather than the prompt. One item missing: nothing notices a dead machine or carries the work elsewhere.*

The first survivor is a file. Run Claude Code headless and it executes the whole agent, prints a result carrying a **session ID**, and exits; hand the ID back with a resume flag and the agent reopens that exact conversation, because the transcript is an ordinary file on disk. Not a toy mode, either: the official GitHub Action ships the same headless agent into continuous integration to review pull requests and turn issues into them.[1][2] The second survivor is the guardrail harness, which matters the moment you hand an agent a shell; every rule in it is enforced by the tool rather than discouraged in a prompt.[3] A line in a prompt is a suggestion; a deny rule is a fact.

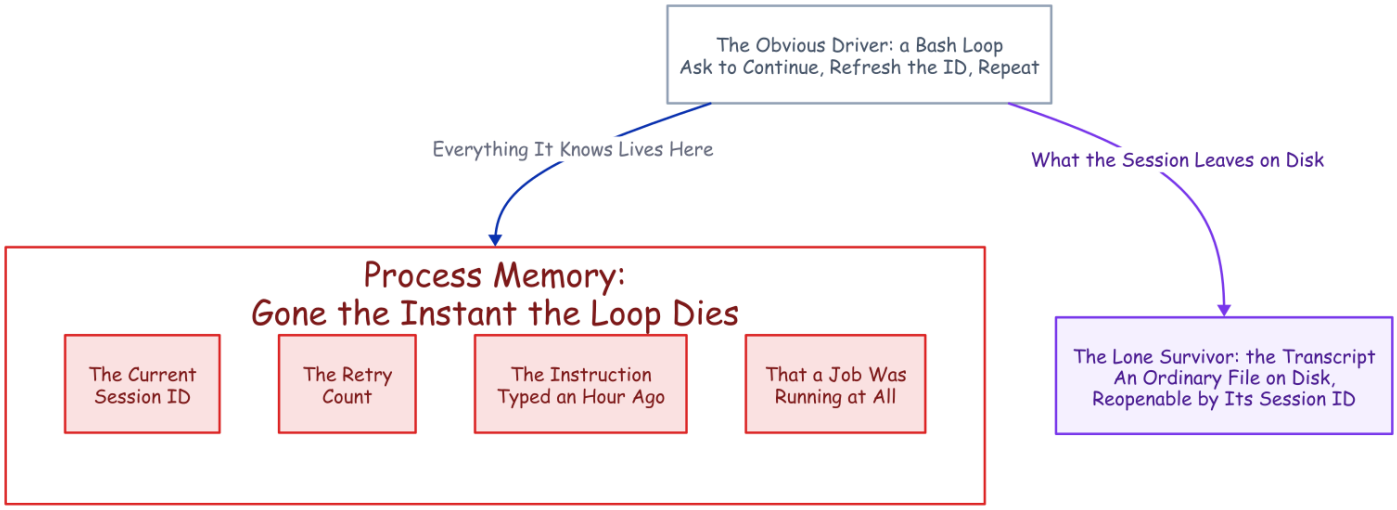
Then the inventory comes up one item short. A resumed session is local to the machine it was born on, a limit the documentation itself concedes.[4] The agent remembers the conversation. **Nothing remembers the job.**

## Nothing Remembers the Job

The obvious way to drive a headless agent is a loop. Ask it to continue, refresh the session ID from the result, ask it to continue again:

```
while :; do claude -p "continue" --resume "$SID" --max-turns 6; done
```

The real thing parses each run's output to keep the session ID current, but that is the shape. Two terms fall out of it. A **turn** is one round of the agent's loop: one model reply plus the tool calls it makes, so `--max-turns 6` stops the run after six rounds. And each capped run is a **chunk**, a bounded slice of the session that stops cleanly with its transcript intact. The loop works right up until the process holding it dies.




*When the loop dies, exactly one thing survives: the transcript on disk. Everything else, the session ID, the retry count, the hour-old instruction, the fact a job existed, was process memory.*

An agent run is exactly the kind of job you cannot afford to forget: it runs for hours and bills by the token, it carries context a fresh start cannot get back, and it fails at two in the morning, in exactly the state you least want to reconstruct from logs. Try to remember all of this yourself and the shopping list writes itself: durable state, a single-writer lock, a retry taxonomy, a liveness probe, a status endpoint. Somewhere around the third item you are hand-building a distributed system you never meant to own. Or you can give the job the one thing the conversation already has: a record that outlives the process.

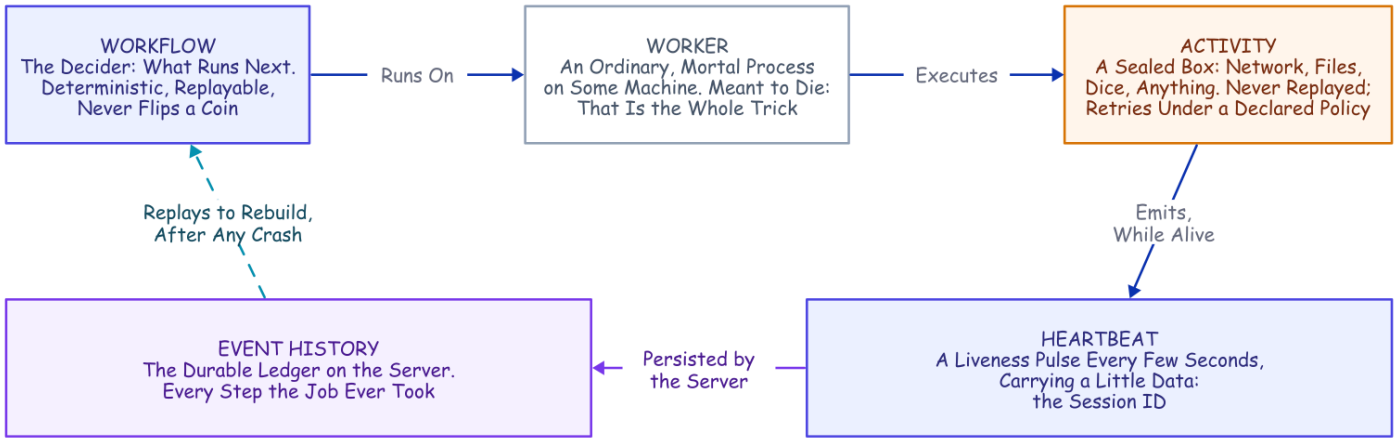
## The Missing Half Has a Name

Strip the agent out of that hand-built list and what remains is not an AI problem at all. It is the standard checklist for any long job that must outlive its own hardware, and the industry has a name for the answer: **durable execution**. Temporal, the engine we used, grew out of a system built at Uber for exactly these long, crash-prone jobs.[5]

The core move is a refusal to trust process memory. A database does not keep your data safe by keeping its process alive; it writes every change to a log and rebuilds after a crash by replaying it. Durable execution does the same for *program control flow*: every step a job takes is appended to a ledger on the server, the **event history**, and when the process dies, a new one reads the ledger back and continues from exactly where the job was.

The reconstruction is **deterministic replay**: re-run the job's code from the first line, substituting the recorded result at every step that already happened. Same code, same inputs, same decisions, same state. The catch lives in the word *same*: one clock reading, one coin flip, one network call in the replayed layer, and the replay diverges from its own history.

That constraint forces a clean split, and four plain words carry the rest of this article:




*The four plain words, and how they connect. A deterministic workflow runs on a mortal worker, which executes sealed activities; a live activity heartbeats the session ID to the server's event history, and that ledger replays to rebuild the job after any crash.*

That last capability, the heartbeat that outlives the attempt that sent it, is the mechanism the rest of this design rests on.[6]

## A Room Where Chaos Is Legal

Now the plan hits the part that looks fatal. A workflow has to be deterministic, and an AI coding agent is the least deterministic software you will ever run: same prompt, different transcript, every time. Put the agent inside workflow code and the first replay diverges on the first token. That is a category error; no knob tunes it away.

But durable execution already has a room where non-determinism is not just tolerated but expected: the activity. Seal the entire agent inside an activity and the workflow never sees the chaos. All it sees is what crosses back as plain data (a session ID, an outcome, a dollar cost, a validated pass/fail report), and its own logic collapses to something a database could run: read a typed result, decide *continue or stop*, repeat.

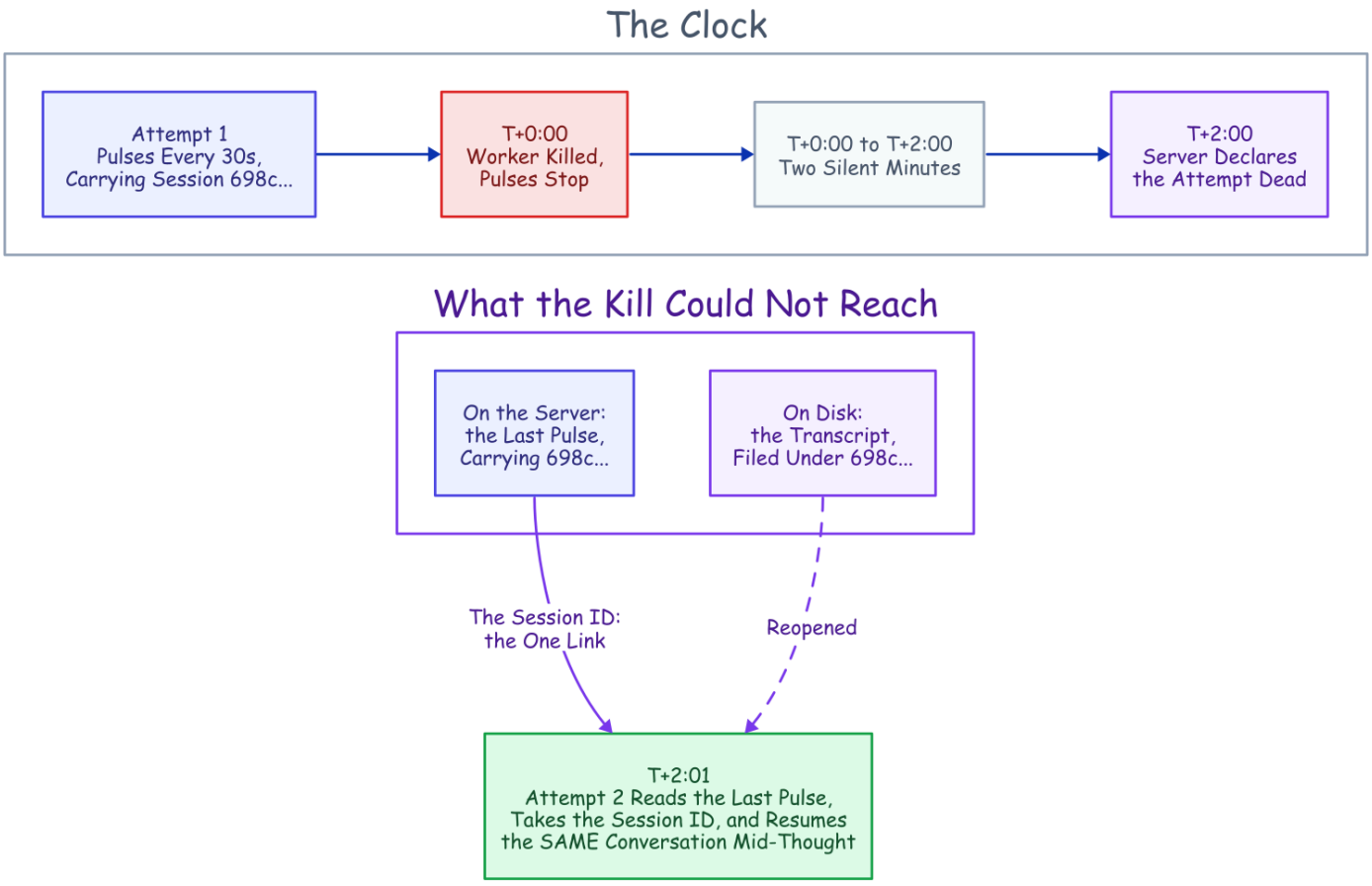
Two details make resume survive that boundary. First, each run gets one stable working directory derived from the job's identity, so every attempt lands in the same folder, where the transcript and the half-edited working tree both live: a retry picks up the conversation and the files it describes. Second, the workflow always chains forward the latest session ID an activity hands back, because a resume can mint a fresh one. The job's ledger stores a *pointer* into the conversation's transcript, and that pointer is very nearly the entire integration. So how does a pointer survive a murder?

## Two Silent Minutes

The whole mechanism fits on a clock. Here is the kill from the top of this article again, frame by frame.

1. **Every thirty seconds, all along**, a timer inside the activity has been pulsing the server: *still alive, and the session ID is* `698c…`. The timer is deliberately dumb: it fires on the clock alone, so a long silent tool call (a slow test suite, a dependency install) cannot be mistaken for death. And how does it know the ID before any result exists? The agent's streamed output opens with an init event that announces the session ID, so the activity learns it seconds after launch, and every pulse afterward carries it.
2. **T+0:00.** Kill the worker and the pulses stop. Nothing was returned; no result was written.
3. **T+2:00.** After two silent minutes, the server declares the attempt dead and schedules a retry on a live worker (in this demo the same worker, restarted, because the transcript lives on that machine's disk; the worker-affinity trick near the end of this article makes that landing deliberate).
4. **T+2:01.** The retry reads the dead attempt's last pulse, takes the session ID out of it, and relaunches the agent with a resume. The agent reopens its transcript and continues from its last persisted line.

Step 4 works because the kill could not reach the two things that matter. Process memory died with the worker, but the transcript lives on disk, the last heartbeat lives on the server, and the session ID in that pulse is the *name* of the transcript. One link, and it is the entire recovery.




*The rescue on a clock. The kill reaches process memory but not the two survivors. The session ID in the last pulse is the one link, and it carries the work to Attempt 2. Color key for the family: indigo is the durable machinery, amber is judgment, purple is the durable record, green is a good exit, red is failure, dashed crosses a boundary.*

No completed checkpoint is required; the last heartbeat is the checkpoint. Every durable-execution engine can resume a *completed* step by replaying its recorded result; this resumes a *live* agent from an attempt that *returned* nothing at all.

And here, at last, is the receipt promised on the first page, one line the recovered run left in its workspace:

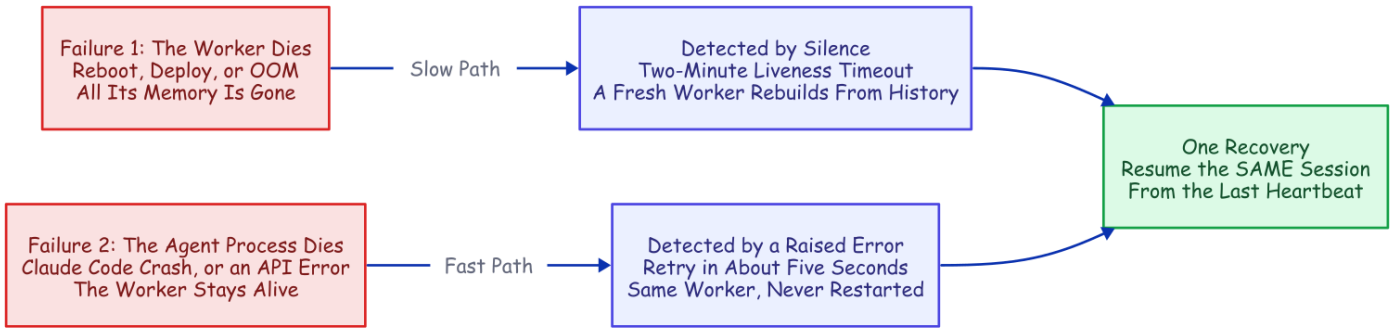
```
{"event": "resume_session_from_heartbeat", "attempt": 2,
 "input_session_id": null, "heartbeat_session_id": "698c432a-…"}
```

The input session ID is null: no completed chunk existed to hand back, so the ID came *only* from the heartbeat. The run finished as the **same** session in one chunk, at $0.0404 on the surviving attempt's meter: there are the four cents, and they are not a recovery penalty, just the resumed session re-reading its own context at the cache rate (the API bills tokens it has already seen at roughly a tenth of the fresh rate) and finishing the work. A replication a month later, on a newer CLI, recovered exactly the same way, heartbeat ID

and all, at $0.1845: identical mechanism, more agent turns, the behavioral spread priced in the economics companion, *Mechanics Cost Cents, Behavior Costs Dollars* (coming soon).

## Two Failures, One Recovery

There are two different ways a run like this breaks, and we killed both on purpose to be sure both recover.




*Two failures, one recovery. A dead worker is caught by silence and rebuilt on a fresh worker; a dead agent process is caught by a raised error and retried in seconds on the same one. Different detection, the same cheap resume from the last heartbeat.*

Both recover the same way; only the detection differs. **The whole worker dies** (a reboot, a deploy, an out-of-memory kill): its memory dies with it, so a *fresh* worker rebuilds the job from the durable event history, and two silent minutes are what reveal the death, the rare catastrophic case durable execution exists for. **The agent process alone dies** while its worker keeps running: smaller, far more common, so it earns its own test. We SIGKILLed the Claude child mid-chunk; nothing global was lost, so the worker caught the broken output stream, retried, and resumed the same session within five to ten seconds across two recorded runs, the worker itself never restarting.

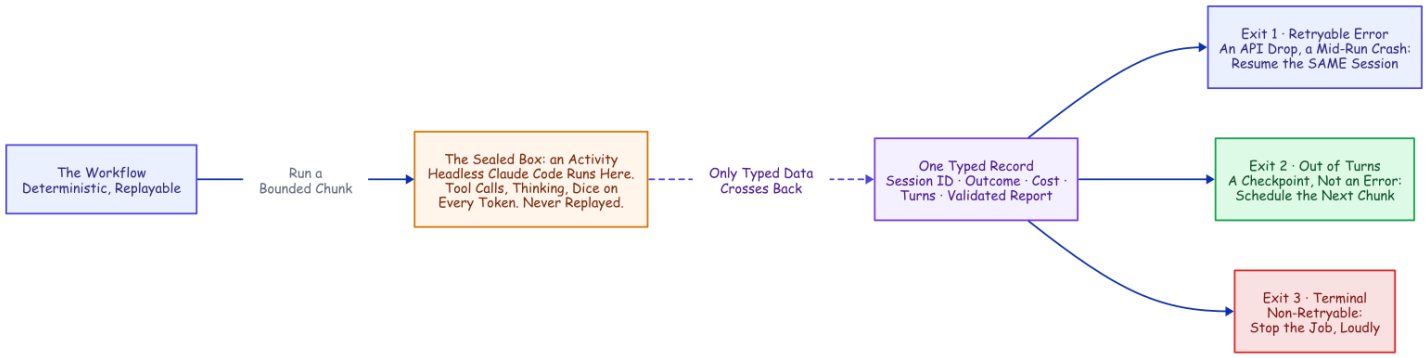
Silence is not the only way to be stuck. The heartbeat timeout catches a silent worker, but a wedged agent pulses forever, so a second alarm, a hard ceiling on the whole chunk, stops that one.

The everyday value is not that deliberate kill; a worker rarely dies outright. What actually stops a headless run is the API: an overload, a rate limit, a dropped stream. The activity raises these as typed, retryable failures; the retry policy backs off (five seconds, doubling to a two-minute cap, six attempts), and every retry *resumes* the conversation instead of restarting it. An overload window becomes added latency and nothing more.

Every failure, from a rate limit to a dead worker, converges on that same cheap resume, inside the sealed box you watched recover. Time to open the box.

## Inside the Box

Open the box and one chunk is a single ordinary function. The workflow calls it, the function runs the agent, and what comes back is a typed record (a session ID, an outcome, a cost, a turn count, the validated report) and nothing else. And the record leaves the workflow exactly three exits, which are nearly the whole control logic: a retryable failure resumes the same session under the declared retry policy; out of turns schedules the next chunk; anything genuinely terminal stops the job, loudly.




*Inside the box. The agent's chaos stays sealed in the activity; the workflow reads one typed record and switches on three exits, outcomes a database could run.*

Launching the agent is a short list of settings: configuration from the project directory only, so the rules ride inside the workspace and the same agent runs identically on every worker; resume the prior session; cap the turns; auto-accept file edits, with every other tool behind an explicit allow-list; and a closing report that must match a schema, so the workflow reads a real pass/fail value instead of parsing prose.

Read the deny rules as architecture as much as safety. Deleting a tree and escalating privileges are the obvious entries; `git push` is the interesting one: the agent writes code but can never reach a remote, so the harness (this project's own Temporal-side code, distinct from Claude Code's built-in guardrails) does every push and merge, in its own step, with its own credentials. The enforcement point is the tool boundary itself: the model never executes a tool, it only *requests* one, and Claude Code, which executes it, consults the deny rules and fires the hooks first, plain programs that return the same verdict for the same call every time. Hooks keep the books and guard the exit: every tool call lands in an audit log, and a Stop hook, the phase gate, refuses to let a run finish until the lane's mandated phases (the backend's run understand, contract, implement, test, self-review, report) all exist as tasks and every one is completed. A mandate here is not a persona; it is an ordered phase list a run cannot skip and still call itself done.

And the policy cannot drift, because the workspace re-stamps it from human-committed code before every chunk, byte-for-byte identical on every attempt; the agent can scribble on its own copy mid-chunk, but nothing it changes survives to the next. None of this needs to be replay-safe, either: hooks fire inside the sealed activity, so the workflow's deterministic replay never sees them. The workflow is deterministic by replay; the hooks are deterministic by construction.

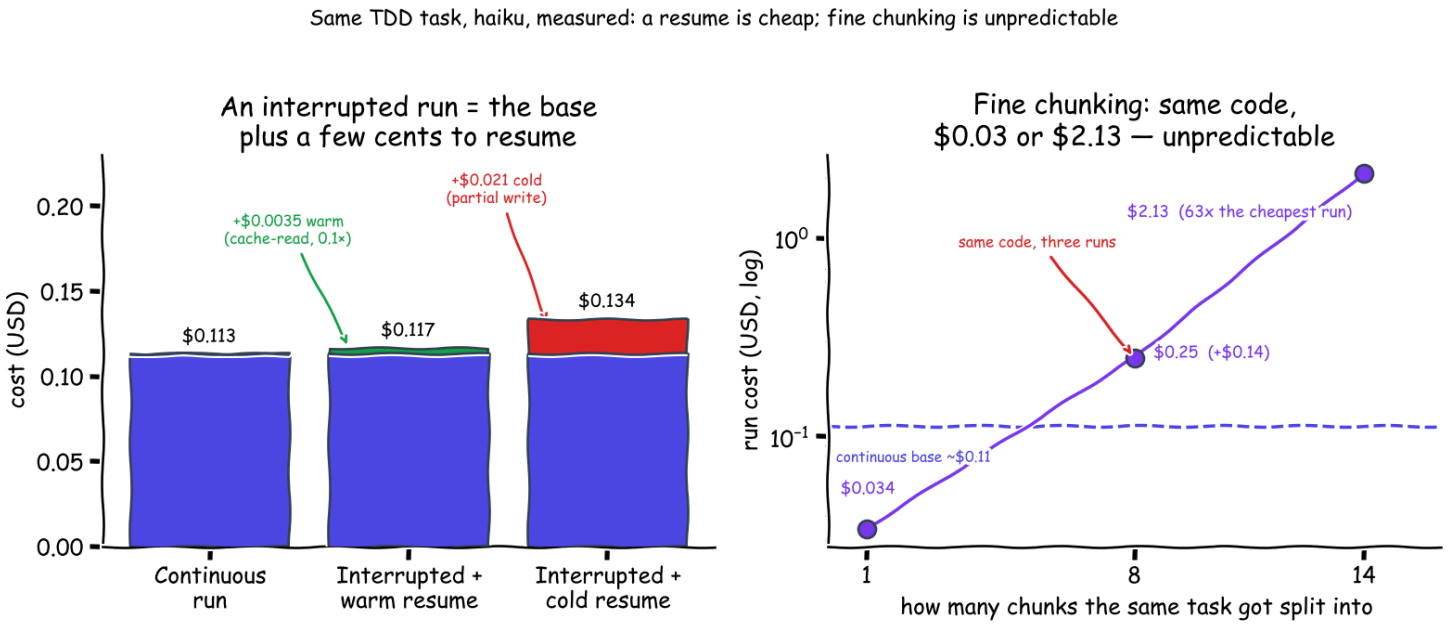
None of this is a bespoke protocol. It is a stock Claude Code project (settings, hooks, skills, memory), assembled fresh every chunk; the bolt-by-bolt detail lives in the engineering companion, How It's Built

(coming soon). This shape was not our first answer. The first answer is worth showing precisely because it *worked*, until measurement talked us out of it.

## Three Numbers and a Slot Machine

An activity's result is all-or-nothing: nothing it learned reaches the history until it returns. So the first design made every turn-cap boundary a **savepoint**: finish a capped chunk, and its progress, cost, and session ID are written into the history. It worked; a savepoint-era kill test, recorded in the project's lab notebook (the experiment log this project distills), recovered on a restarted worker and finished all ten of its tests, one session end to end. But once heartbeat recovery existed, those completed-boundary checkpoints were buying a durability we already had, and each one was another resume. (The full detour, savepoint diagram and all, is in How It's Built (coming soon).) Worse, they were quietly doing something to the cost, something we did not see coming.

So we measured the whole bill, holding one task constant across every scenario on the small fast model, observing everything rather than modeling it, strictly one run at a time. Three numbers carry the result.




*The measured bill, one test-first task on haiku. Left: surviving a crash is the base cost plus a few cents, warm or cold. Right, log scale: the same code split fine ran $0.034 to $2.13, a spread that is behavioral, not a cache tax.*

Two of those numbers reassure and one warns. On these runs, surviving a crash was almost free, a cache re-read rather than a re-run, warm or cold. The warning is fine chunking, and the culprit is not the per-boundary cache re-read (a cheap fraction of a cent) but **behavior**. A tight turn cap changes what the model does with its turns, and a run that wanders across fourteen chunks and twenty-eight turns, to finish what a continuous session did in nine, pays for every wander. (An earlier experiment had even sworn chunking was cost-neutral; the correction, the method, and the pricing canary that re-checks every number on a schedule are the economics companion, *Mechanics Cost Cents, Behavior Costs Dollars* (coming soon).)

So the rule is large chunks by default: fine chunking is the only lever that meaningfully moves the total, and it moves it unpredictably. One pattern runs through every number in this section and everything else this project measured: **the mechanics cost cents; the behavior costs dollars.** Small chunks, though, earn their keep for a different job entirely.

## Query, Steer, Cancel: What the Boundaries Became

Completed-chunk boundaries did not stop being useful when heartbeats took over the durability. They were reborn as the *visibility and steering* cadence: the moments where a running job can answer to verbs a bare process cannot.

**Query it.** A read-only question, answered from the workflow's own state rather than from logs. Mid-run, a status query comes back with the chunks completed so far, the running cost, and the latest chained session ID (mid-chunk, the live ID rides in heartbeat details; workflow state catches up at the boundary).

**Steer it.** A **signal** is Temporal's message to a running workflow, no relation to the Unix signal that killed the worker earlier. One lab-notebook demo injected *"also add a stats subcommand, same test-driven treatment"* into a running job, and the workflow folded the instruction into the next chunk's prompt. And if a steer arrives during the very last chunk, it triggers one more, up to the job's own chunk cap, so late guidance is not silently dropped.

**Cancel it.** Delivered on the next heartbeat: the activity tells the running agent to stop, so it exits cleanly instead of burning tokens as an orphan, and the job lands in a canceled state you can see.

So the turn cap has one clear rule: make it small only when a human needs to watch or steer in near-real time. And the left column of this table is the hand-built shopping list from the start of this article, paid off item by item, plus one item you would never have gotten to:

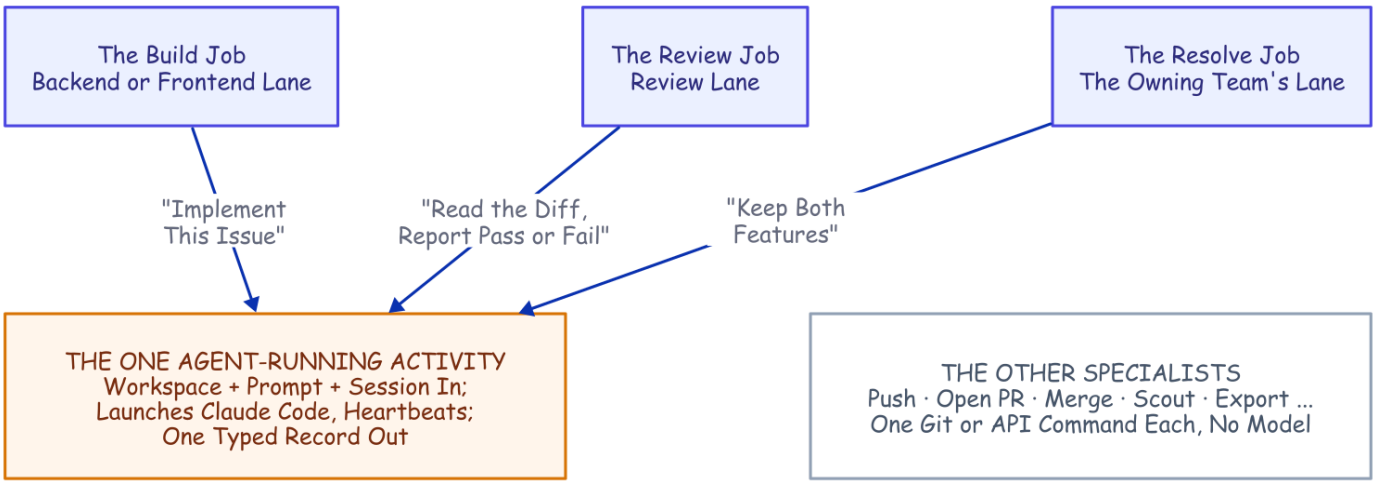
| What you would build by hand | The primitive that already exists |
|---|---|
| A session-ID file plus a lockfile | Job-ID uniqueness: one job *is* the session's single writer (workflow-level; attempt-level fencing stays on the gaps list) |
| A restart script plus a liveness probe | A heartbeat timeout plus a retry policy, declared not coded |
| A task-to-session map in memory and on disk | The session ID in heartbeat details mid-run, and in workflow state after a chunk |
| A hand-rolled taxonomy of which errors to retry | Typed retryable vs. non-retryable failures, with backoff |
| A status endpoint scraped from logs | A progress query plus the event-history UI |
| Steering a running task (never actually built) | A signal, folded into the next chunk's prompt |

Point that same machinery at a whole repository and it does something bigger. The heartbeat, the resume, and the declared retries that just carried one agent through a crash are what will carry a whole *team* of single-purpose durable jobs through the issue-to-merge run promised at the top, conflict and all. That team is the rest of this article.

## Nine Specialists, Two Conductors

Everything so far makes *one* job durable: the one that runs the agent. A durable job, unlike a bare process, can be given an owner, an address, and rules. So the team is not that agent cloned into a crowd; it is more single-purpose durable jobs built around that agent, each with its own mandate. A nine-member team is nine durable jobs under one protection, and only the one that calls a model is an agent at all. The split follows one rule: **judgment goes to the model; mechanics go to code.**

Here is the layering that makes one agent go a long way. The agent-running activity is a single ordinary function, and it knows nothing about backend or frontend: it takes a workspace, a prompt, and a session to resume; launches Claude Code; heartbeats; and hands back a typed record. **A role is a set of arguments**: build, review, and resolve are different team members only because of what surrounds the call. The next section makes those surroundings (the queues, the namespaces, the lanes) precise.



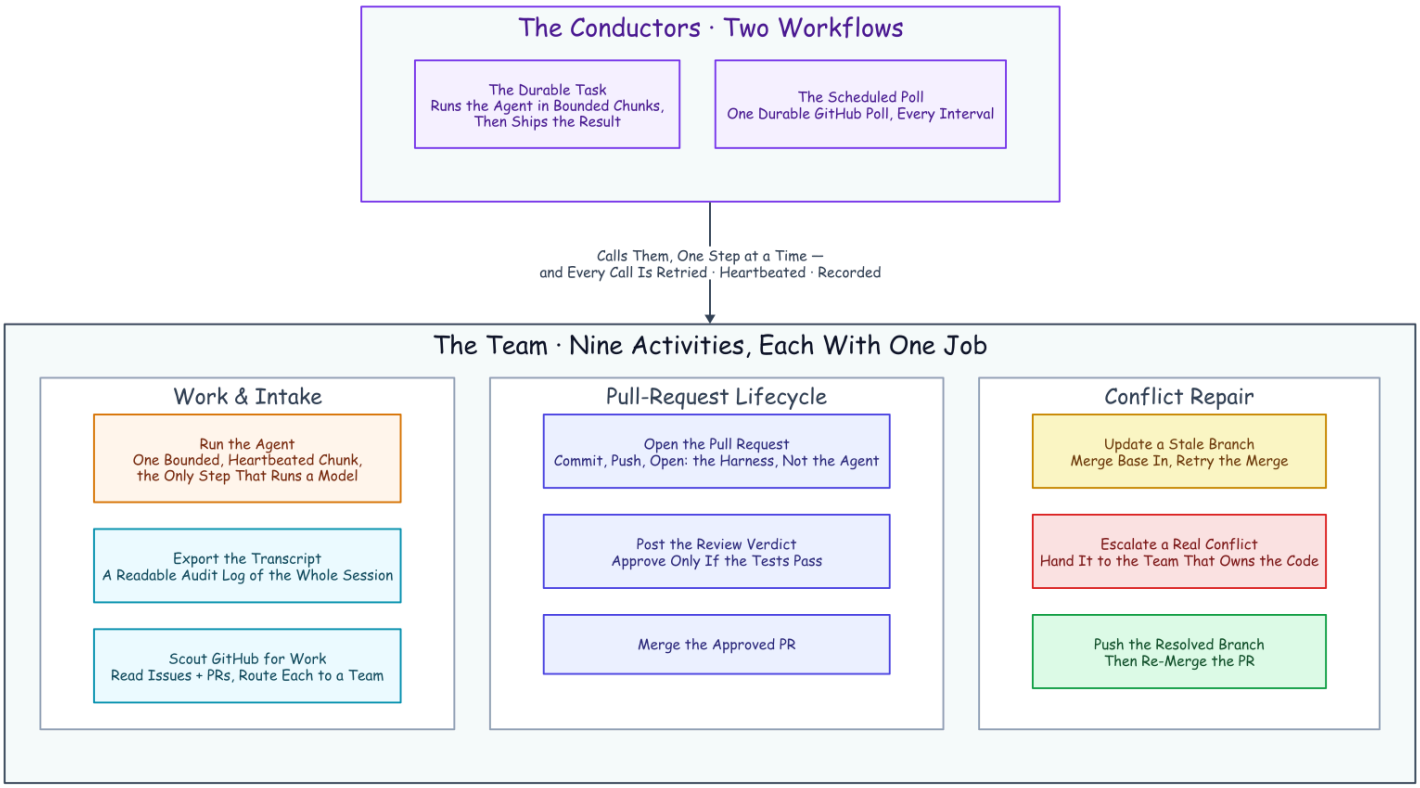

*One function, many roles. The build, review, and resolve jobs each hand the same sealed activity a different prompt from a different lane; every other specialist is one git or API command with no model in it.*

| **The argument** | **"Backend engineer"** | **"Reviewer"** | **"Conflict resolver"** |
|---|---|---|---|
| The function called | Run the agent | Run the agent | Run the agent |
| The prompt | Implement this issue, test-first | Read the diff, run the suite, report pass or fail | Merge main in, keep both features |
| The skills in the workspace | Backend playbooks | Review playbook plus the verdict schema | The owning team's playbooks |

| The argument | "Backend engineer" | "Reviewer" | "Conflict resolver" |
|---|---|---|---|
| The queue and namespace | Backend | Review | Backend, the owning team's |

The callers also decide *when* judgment is needed, and most of the time it is not. The build job calls the agent for every chunk, because writing code is all judgment. The review job calls it once. The merge job never calls it: merging an approved pull request is one API call. Updating a stale branch sits on both sides of the rule: a mechanical re-merge right up until a content conflict escalates it to an agent, a moment the conflict story below earns in full. Every other specialist is one mechanical act, a git command, an API call, a transcript export, and a git command does not need a model.

Here is what we actually built. **Two workflows conduct the delivery team.** One is the durable task you have already met: it runs the agent in bounded chunks and then ships the result. The other is a scheduled poll that looks at a repository for new work, on a timer that is always counting down toward its next sweep, as a durable job in its own right; keep that timer in mind, because it quietly drives everything that follows. And beneath those two conductors sits a team of single-purpose activities (nine in the runs measured here; twelve today), **each with exactly one job.** None of them is clever. Each is a sealed box that does one real-world thing and hands back typed data, and that narrowness is the point: it is what lets the retries, the heartbeats, and the audit trail apply to every one of them for free.




*The team: two conductor workflows call nine one-job activities, and every call is retried, heartbeated, and recorded. Only the amber box runs a model; the rest carry the pipeline around it.*

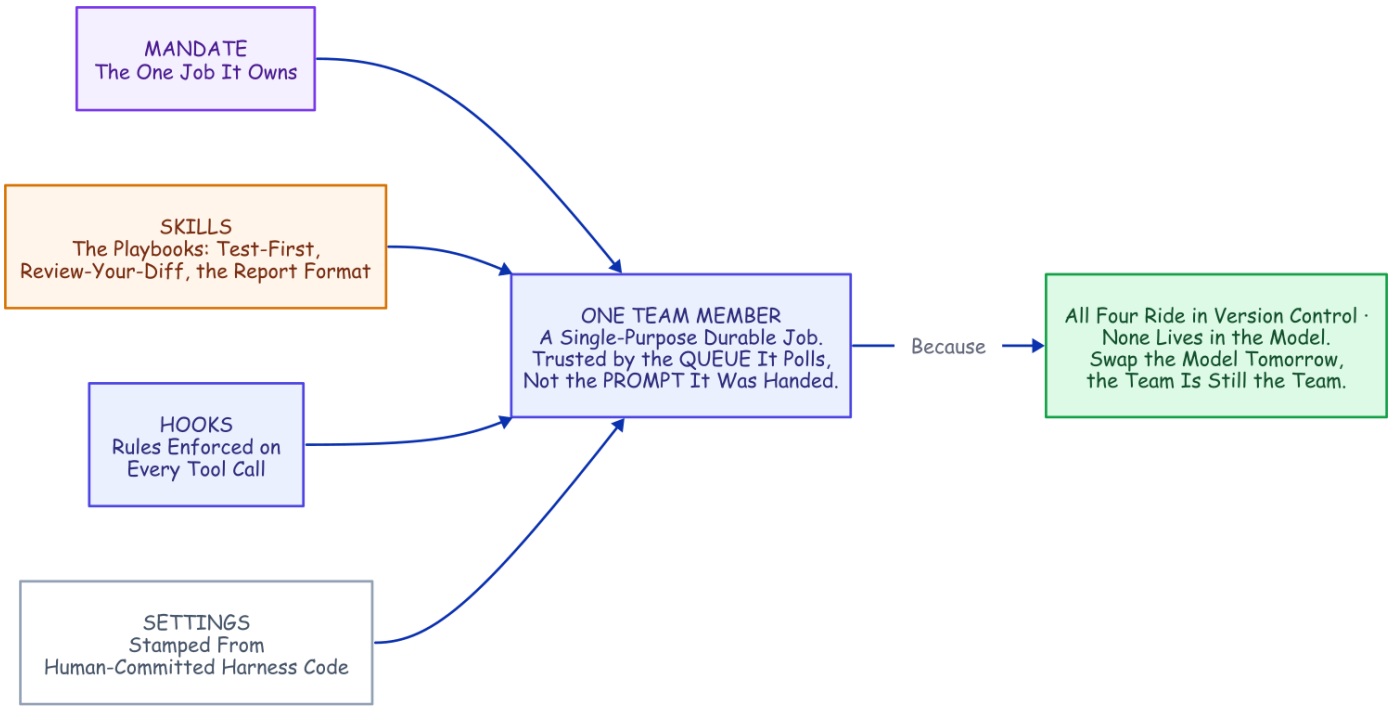
Nine specialists, two conductors: a filed issue can travel the entire distance to a merged pull request while the humans sleep. The roster above names each member by its one job, in the three groups that carry an issue: the work itself, the pull-request lifecycle, and the repair path for when a merge fights back. The shape should feel familiar: it is the discipline of a hardened CI/CD pipeline (a protected trunk, gated merges, least-privilege credentials, an audit log on every step) applied to a team whose members are durable jobs.

The reason this deserves the word *design* is that adding to the team is mechanical. Want a new capability (a security scan, a changelog write, a deploy)? You write one function that touches the real world, you declare how it should retry, and you name the moment in the workflow where it runs. It inherits crash recovery, backoff, cancellation, and the audit trail straight from the harness. A new *agent* role costs even less: a new lane, a new skill bundle, a new prompt, and zero new model code, because the one hardened activity does the running. That is the whole trade: write the happy path and rent the rest.

## Trusted by the Queue, Not the Prompt

The team scales by giving each *kind* of work its own lane, and every "team" noun here is a concrete primitive underneath. A **namespace** is an isolated partition of the Temporal server. A **lane** is this article's word for one team's namespace plus everything the team owns inside it: its own work queue, and its own bundle of skills bound into the workspace before each chunk runs. The teams in these runs were backend, frontend, and review; the system runs six lanes today. Workers listen only to the lane they own.

A backend worker is trusted to do backend work not because its prompt says *act like a backend engineer*, but because it listens on the backend queue and runs under the backend team's committed playbooks. **It is trusted by the queue it polls, not by the prompt it was handed.** The mandate itself is an ordered phase list, worked the same on every run, and the phase gate holds every run to it. The load-bearing rule inside that skill bundle: a change in one layer has to prove it did not break the others before review will approve it.



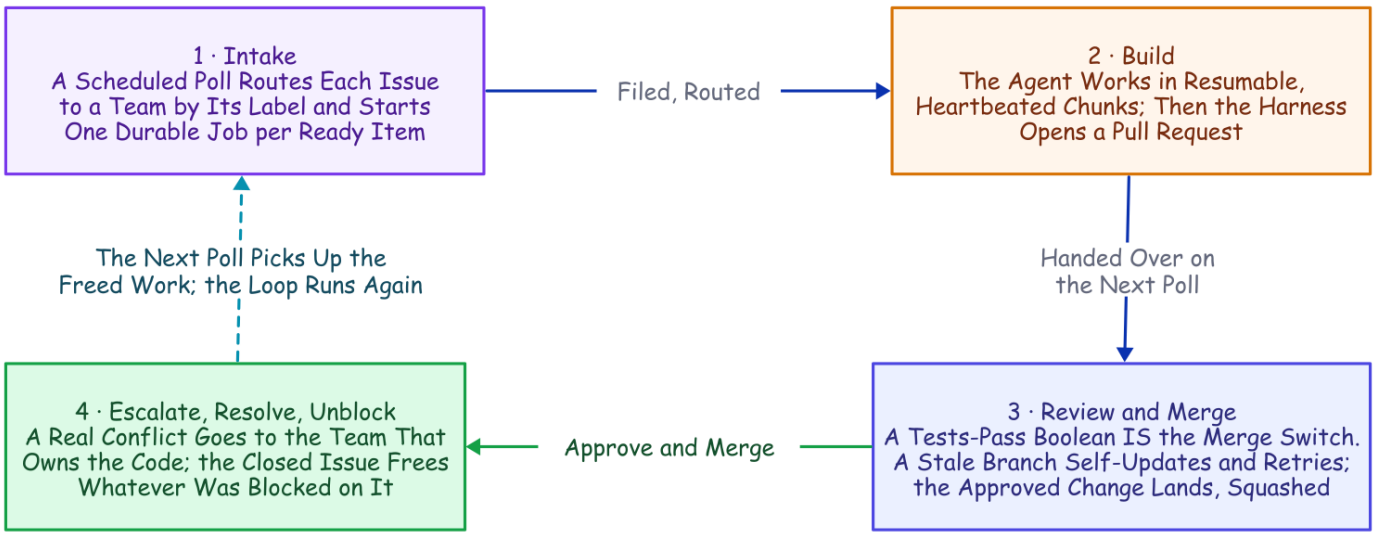

*The formula for one team member: a mandate (an ordered phase list the phase gate enforces), skills, hooks, and a settings file stamped from committed harness code. All four ride in version control; none of them lives in the model.*

You could swap the model tomorrow and the team would still be the team.

One more primitive and one structural catch finish the picture. **One owner per piece of work** comes from deriving each job's ID deterministically from the issue and starting it with an "allow duplicates only after failure" policy: a second attempt at a running or completed job is rejected by the server, while a failed job may retry on a later sweep. That single fact is *why* the poller can be completely stateless, and why a transient failure heals itself instead of deadlocking an issue forever. A workflow can only start child workflows inside its own namespace. So when the intake or an escalation has to cross a lane, it goes through an activity instead: an activity may do arbitrary I/O, so it opens a client to the target namespace and starts the job there. Remember that move; you are about to see it twice.

## One Issue Walks the Whole Loop

Put the conductors and the team together and one issue travels a full loop, every box along the way a durable step in the history.




*One issue, the whole loop. Every box is a durable step; the conflict detour and the unblock are not special cases, just the same machinery running once more.*

**Intake.** A schedule fires the poll on an interval. The poll reads the repository (open issues, open pull requests, which issues have closed), routes each item to a team by its label, and starts one durable job per ready item with a deterministic ID: the first of the two client-from-inside-an-activity crossings. An issue marked *blocked by* another is held until that other issue closes.

**Build.** The issue's job runs the agent in bounded, resumable chunks, heartbeating its session ID the whole time; the agent commits as it works, closing each phase with a commit, so the branch accumulates a real history of checkpoints rather than one anonymous blob. And each commit is mirrored to the remote work branch the moment it lands (a hook leaves a marker; the harness pushes, never the agent; a rule added after the recorded runs), so none of that history exists only on the worker. When the agent reports success, the transcript is exported to a readable audit log and a pull request is opened from the already-mirrored branch, carrying that full commit history.

**Review and merge.** The open pull request is picked up on the next poll and routed to the review lane as its own job. The review agent reads the diff and runs the suite, and its verdict is a single field in the required report: a pass/fail boolean that *is* the merge switch. True posts an approving review comment and merges; false posts a changes-requested comment, and in the measured runs the trail ended there: the bounded fix loop that now routes a denial back into a fresh build round arrived later (*The Human Is a Durable Object* (coming soon) tells it). A merge against a branch that has fallen behind triggers an automatic branch update (the same "update this branch" move you would click in the GitHub UI) and a retry.

**Escalate, resolve, unblock.** A retry that *still* conflicts is a genuine content collision, and it gets handed to the team that owns the code: a resolve job in that team's own namespace, where an agent fixes the conflict by hand and runs the tests, and the harness pushes and lands the merge. When that merge closes the issue, the next poll releases whatever was blocked on it, and the chain runs again for the dependent.

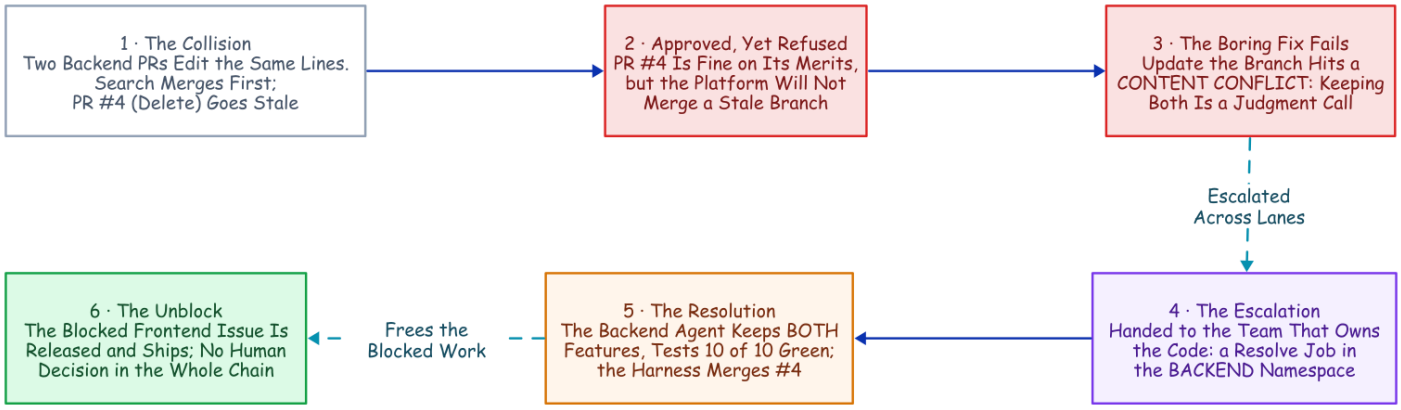
That is the happy path, walked once. The proof that the team is real is the day the path broke.

## The Conflict That Resolved Itself

Every piece of that machinery was about to be needed at once, so we built a real target for it: a full-stack "snippets" web app, backend and frontend and browser tests, on a live deployment with a namespace per team and the poller on a schedule.

Two backend issues landed in the same poll window and collided by construction: one added search, the other a delete endpoint, both editing the same lines of the same backend file. The search PR merged first, and the delete PR (**PR #4**, the one this story follows) went stale. This is where autonomy usually ends quietly: a "needs a manual rebase" comment, and a human inheriting the mess in the morning.

Here is what happened instead, every step a durable activity in the history.




*PR #4, end to end. Two PRs collide, the delete one goes stale, the update hits a content conflict, and the review lane hands it across lanes to the backend team; the agent keeps both features and the harness merges, which releases the frontend that had been blocked all along. Every box is a durable step.*

The figure is the whole chain; only one step in it needed a mind. The review lane approved #4 on its merits, but the platform will not merge a stale branch, and the boring fix (merge main back in, retry) hit a **content conflict**: both PRs had edited the same lines, and no re-merge decides which side wins. Someone has to read both changes and write the version that keeps both. That is a judgment call, and judgment is what the agent is for. So the review lane handed it to the team that owns the code, through the same client-from-inside-an-activity call the poll uses; the agent that picked it up was nothing special, same skill bundle and policy and harness, only the prompt differed.

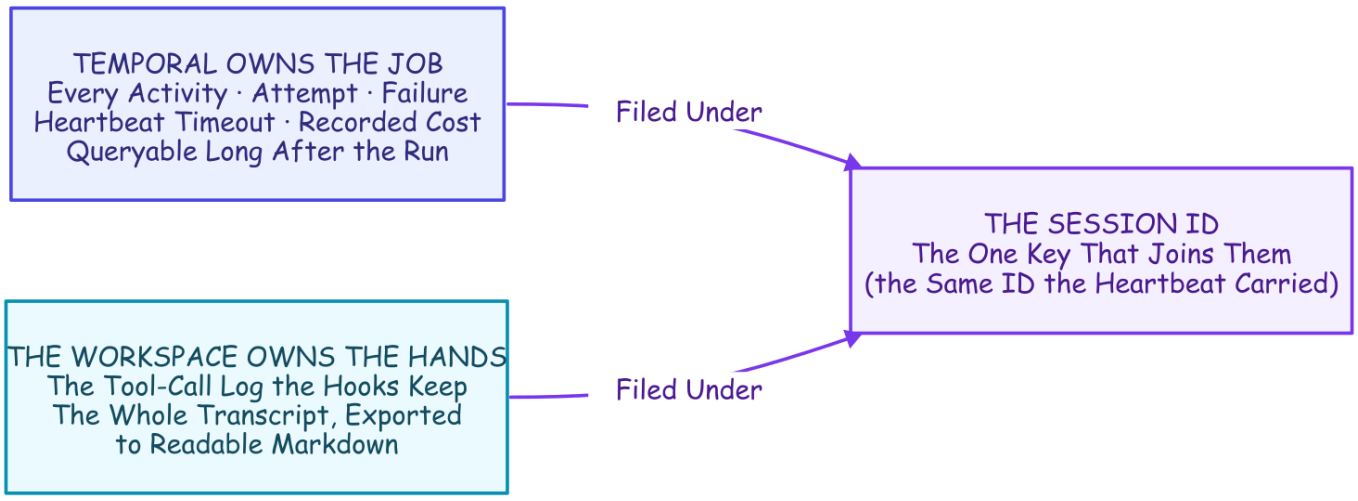
Two properties keep this from being a party trick. The escalation cannot loop: the resolve job's ID is deterministic (today it derives from the PR and its current head commit), so a duplicate start is refused by the server and cascading conflicts converge instead of multiplying. And it cannot cheat: the escalation is just a workflow starting another workflow, under the same crash recovery and hard limits as everything else.

Then the part that turns an incident into a system, the last box: a frontend issue, marked *blocked by* the backend one, had waited the whole time. The next poll released it, and the frontend agent shipped a delete button and a browser test against the endpoint the resolution had just landed. From collision to shipped feature, no human decision in the loop, one mechanical asterisk included: this run reconstructed the chain rather than waiting for it, the review re-triggered against the new escalation code (PR #4 predated the feature), and the polls fired by hand rather than left to the timer. The scheduled poll and the first-pass routing are proven across the other recorded runs; this run isolates the resolution chain.

A claim that size earns a question: how would you even know it is true? You would read the record, and every run leaves one.

## Every Run Leaves Evidence

Because the agent runs under a policy the workspace owns, every run leaves *two* planes of evidence, joined on one key: the session ID, the same ID that rode the heartbeat on the first page.




*Two planes, one key. Temporal owns the record of the job; the workspace owns the record of the hands. The session ID files both, so a cost in one plane and a tool call in the other belong to the same run.*

Collect those exports and you have a corpus of how your agents actually behave, shipped to a bucket, one folder per run. And a corpus is a feedback loop waiting to be closed, so we closed it. A reviewer read the tool-call logs across the cost-experiment runs and found one consistent waste: the agent kept re-checking work it had already done, listing a directory to confirm a file it had just written, re-running tests that were already green. We codified the lesson two ways, a written rule and a deterministic hook, and re-ran. The rule bought a modest ~20% drop (half an instance per run, inside run-to-run noise) and one run ignored it entirely; the hook flagged every remaining instance, twelve out of twelve. **A line in a prompt is a suggestion; a hook is a law.** And a mined corpus is how the laws get written.

The loop has held since: on an overnight run of the full team, the audit trail those hooks write was the record that debugged five live failures into five same-evening commits, one of them the dedup-deadlock fix you will meet in the defaults below. The system's job is not to avoid failure; it is to convert failure into a commit.

## What This Doesn't Solve

Five real gaps, told plainly, because the honest ones are the load-bearing ones.

- **The filesystem is not checkpointed, and steps run at least once.** A retry resumes against a directory the dead attempt half-mutated; every test converged (even a torn transcript resumed with full context, in a staged run), but the real fix, a fresh git worktree per chunk plus idempotency keys on outward actions, is not built.
- **A permanently dead machine takes the live session with it.** The commit mirror bounds the code loss (whatever was committed is on the remote, and a fresh worker's clone starts from it), but the conversation and anything never committed still die with the machine. A two-filesystem run has proven the resume off shared storage (How It's Built (coming soon) has the receipt), but the harness still does not ship the sync itself.
- **A hard kill can orphan the agent.** The child may finish its chunk anyway, racing the retry, and a paused worker triggers the same race with no kill at all; kill by process group, and know that the fencing check that would close the race is not built.
- **Every input is trusted.** Issue text, PR bodies, and code comments are all prompts, so a public deployment is an injection surface at each one; of the three mitigations (author allow-lists, injection-aware review, a human gate), only the gate exists, built but off by default.
- **The merge gate is an agent's verdict.** One tests-pass boolean carries the merge, and it ran wrong three in ten when agents graded their own fresh fix; the case a real merge rides on, an honest verdict on genuinely passing work, is still unmeasured (*The Agent Grades Its Own Homework* (coming soon)). Denying `git push` does not close the chain either, because the agent shapes both the artifact and the verdict that make the credentialed harness act; for production, put your human gate there (*The Human Is a Durable Object* (coming soon)).

## Defaults Worth Stealing

If you build any version of this, on any stack, these are the settled defaults this project would hand you. Each one was paid for above.

- **Big chunks by default.** A tight turn cap is a slot machine: the same task ran $0.034 to $2.13 on identical code. Chunk for visibility and steering, never for durability.
- **Heartbeat a resume handle, not just liveness.** The last pulse is a free checkpoint; a retry that can read it never starts from zero.
- **Kill, and deploy, by process group.** An orphaned agent child finishes the work anyway and lies to you about whether your recovery works.

- **Deny `git push` to every agent, then mirror every commit for it.** The agent writes code and commits as it works, but it can never reach the remote: on each commit a hook leaves a marker, and the harness mirrors the work branch in its own step, with its own credentials, so the ledger is never only local. Pushing and merging stay separate recorded steps in the durable history; the pull request carries the agent's full commit history (the merge itself lands squashed by default). No agent holds a world-changing credential, and every outward change has an audit entry: judgment and credentials never share a process.
- **Let the platform own the trunk.** Branch protection is the governance layer that holds even if everything above it misbehaves: main takes pull requests only, nobody pushes it directly (agent or harness alike), force pushes are refused, and the merge stays an API call on a PR, recorded like everything else. One script turns it on, and its review-count dial (zero keeps the loop autonomous, one is the human gate) is exactly where production tightens.
- **Route by queue, not by prompt.** A lane's queue and skill bundle are an identity a worker cannot drift out of; a persona in a prompt is not.
- **Wake the model only for judgment.** The boring fix goes first: a stale branch is a mechanical re-merge, and only a content conflict escalates to an agent. Most days the mechanical path wins, in milliseconds, for free.
- **Pin the resume to the worker that holds the session.** Opt-in worker-affinity queues (the harness's own routing, distinct from Temporal's built-in sticky execution): the first chunk runs on the shared lane queue and reports its worker's own stable queue; the workflow pins every later chunk there. A restarted host resumes correctly, and a chunk never silently lands where the session is absent.
- **Deterministic job IDs, duplicates allowed only after failure.** Running and completed jobs are refused, so the poller needs no memory and the escalation loop cannot run away. Failed jobs may retry on a later sweep, so one transient error cannot deadlock an issue forever. (A live fleet run found the stricter refuse-everything version of this rule doing exactly that; the evidence archive has the receipt.)
- **Write the rule as a hook, not a prompt.** Measured head-to-head, the rule was always probabilistic and the hook always law, down to a rule losing outright to a direct instruction while the hook caught twelve of twelve. Priced at the tool boundary in *Flag, Block, or Beg* (coming soon) and at the finish line in *Done Is Not a Claim* (coming soon).

## The Harness Was the Hard Part All Along

Most of what an agent system needs is not the AI at all; it is the operational harness: permissions, state, recovery, tools. Set that against the hand-built list from the start of this article (and the table that mapped it, item by item, to existing primitives) and the overlap is hard to miss. The harness every agent team is rebuilding is, almost line for line, what durable-execution engines have provided for a decade.

The two sides are already circling each other. Temporal's own AI cookbook wraps model *API calls* in activities; a production write-up wraps the agent framework the same stateless way; another project built durable checkpointing around whole Claude Code runs on its own runtime.[7][8][9] In the sources we checked,

none composes the two the way this project does: headless Claude Code *sessions*, the session ID carried in heartbeat details mid-chunk and in workflow state after a completed one. That claim has a short shelf life, so re-check it before you build.

Claude Code brings the durable conversation. Temporal brings the durable job. The seam between them is one pointer: a session ID, carried in a heartbeat, chained through a ledger. That is the whole composition, and it is why the kill on the first page is staged as a stunt and lands as the boring case. The agent could always remember the conversation. The merged pull requests, the conflict that resolved itself, the failures converted into commits: all of it is what became possible the moment something finally remembered the job.

## The Family

This article is the trunk; six companions each own one branch, and they read in this order:

- How It's Built (coming soon), the rivets: the settings, the tuning, the lane plumbing, the details the story above skips
- Mechanics Cost Cents, Behavior Costs Dollars (coming soon), the bill: every boundary priced, the method, and the canary that re-checks it on a schedule
- Flag, Block, or Beg (coming soon), the tool-call boundary: what a prompt, a flag, and a block each buy, measured
- Done Is Not a Claim (coming soon), the finish boundary: the same hard deny, moved to the exit, inverts the result
- The Agent Grades Its Own Homework (coming soon), the verdict boundary: the merge switch's boolean against ground truth
- The Human Is a Durable Object (coming soon), the person: the one human decision, modeled as durable state, deny-safe on silence

---

*Disclaimer: the views and opinions expressed in this article are my own and do not necessarily reflect those of my employer. This is a personal project, not affiliated with or endorsed by Anthropic or Temporal.*

## Sources

- **Vendor documentation** for every Claude Code behavior cited (headless mode, sessions, the CI action, permissions, hooks, sandboxing) and every Temporal behavior (durable execution, activities, heartbeats and their throttling, versioning): code.claude.com/docs, docs.temporal.io, python.temporal.io; specifics footnoted inline. *Vendor/canonical.* Checked 2026-08-20.
- **Practitioner and adjacent sources** (the Temporal AI cookbook and the two adjacent durable-agent write-ups) are footnoted inline.

---

1. Claude Code headless mode: `--output-format json` returns `session_id`, cost, and (with a JSON schema) structured output; `--resume <id>` continues a stored session; transcripts are stored under `~/.claude/projects/<cwd-slug>/<session-id>.jsonl`, the working-directory path with non-alphanumerics replaced. Checked 2026-08-20. *Vendor/canonical.*↩

2. `anthropics/claude-code-action@v1` (GA) runs headless Claude Code from GitHub workflows: it activates on `@claude` mentions, issue assignment, or whatever events the workflow declares, `pull_request` and cron included, and lists automatic PR review and turning issues into pull requests among its documented use cases; its configuration is aligned with the Claude Code SDK. Checked 2026-08-20. *Vendor/canonical.*↩

3. Claude Code permissions are evaluated deny → ask → allow (a deny is unoverridable) and "enforced by Claude Code, not by the model"; `PreToolUse` hooks can deny a call before it runs, while `PostToolUse` hooks receive the completed event as JSON and report back; Bash sandboxing uses Seatbelt (macOS) / bubblewrap (Linux). Checked 2026-08-20. *Vendor/canonical.*↩

4. Claude Code sessions: since CLI v2.1.223 the resume lookup searches the current project directory and its git worktrees first, then every other project on the same machine (before that release it stopped at the project directory). Either way there is no documented cross-machine session transport or leasing: the gap an external orchestrator must fill. Checked 2026-08-20. *Vendor/canonical.*↩

5. Temporal docs: durable execution persists workflow progress as an event history; workflows must be deterministic so history replay reconstructs state; activities host side effects and are retried by policy; Temporal descends from Cadence, built at Uber. *Vendor/canonical.*↩

6. Temporal Python SDK: `activity.heartbeat(*details)` sends heartbeat details, and `activity.info().heartbeat_details` exposes the previous attempt's details to a retry. The worker throttles how often details are persisted (`max_heartbeat_throttle_interval` / `default_heartbeat_throttle_interval`). Checked 2026-08-20. *Vendor/canonical.*↩

7. Temporal AI cookbook: the "Basic agentic loop with Claude" uses model Messages API calls as activities; no Claude Code CLI, no session resume. *Vendor/canonical.*↩

8. "Building fault-tolerant long-running AI workflows with Claude Agent SDK × Temporal.io", claudelab.net (2026-04-22): wraps API messages in activities; no headless sessions, no cross-activity resume. *Practitioner.*↩

9. "Don't make Claude do the same work twice", zenml.io (2026-06-01): durable checkpointing of Claude Agent SDK runs on ZenML's own runtime, not Temporal. *Practitioner.*↩